

Computational cost and reflections on sustainable AI

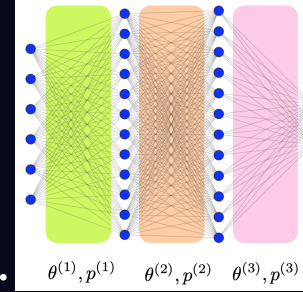
Dr Tiffany Vlaar
University of Glasgow

VIASM Autumn School Lecture
September 2026

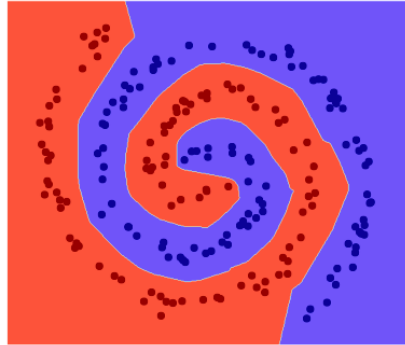
Try to enhance our understanding of neural networks

And use these findings to reduce computational cost

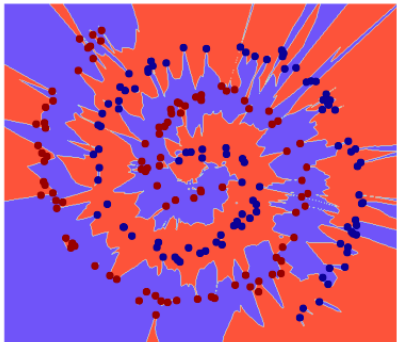
while enhancing or maintaining generalization.



Generalization



(a) 100% train, 100% test



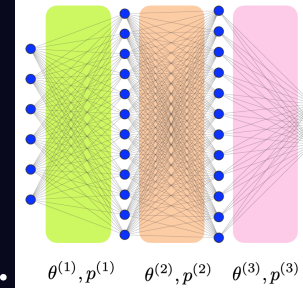
(b) 100% train, 7% test

Huang et al., 2020

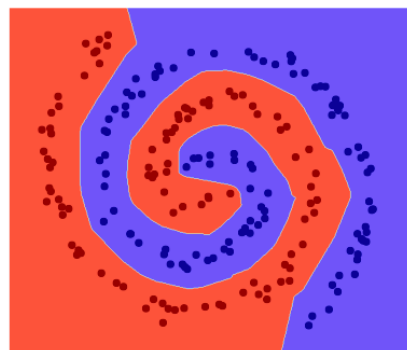
Try to enhance our understanding of neural networks

And use these findings to reduce computational cost

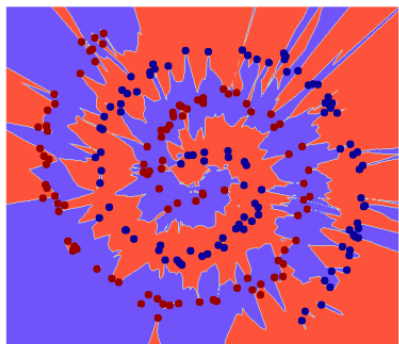
while enhancing or maintaining generalization.



Generalization



(a) 100% train, 100% test



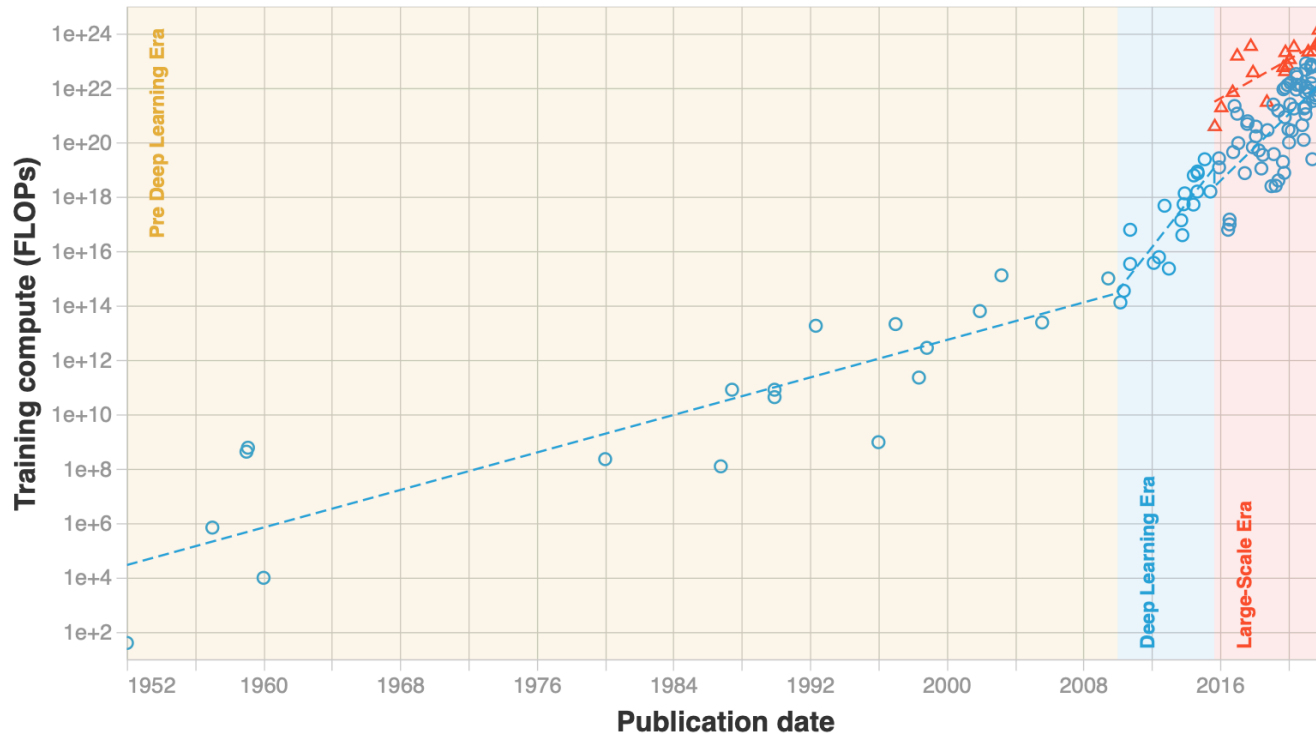
(b) 100% train, 7% test

Huang et al., 2020

Computational Cost

Training compute (FLOPs) of milestone Machine Learning systems over time

n = 121

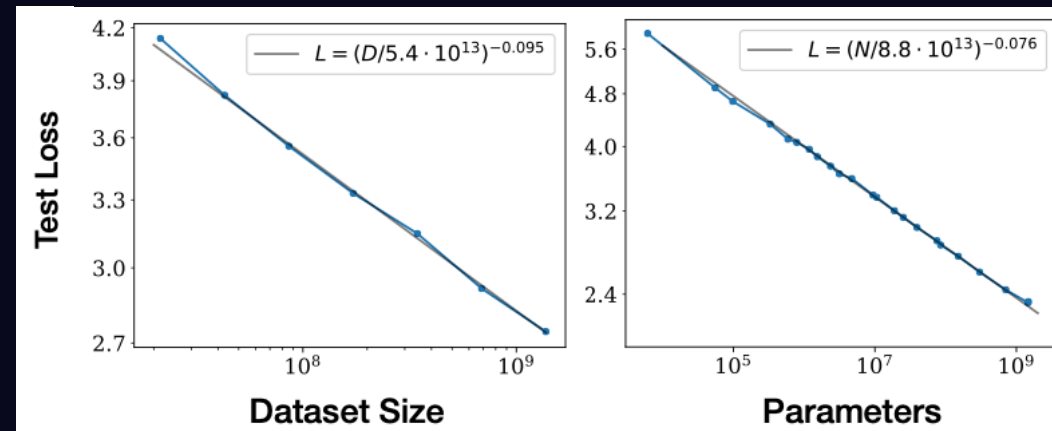


Sevilla et al., 2022

Model size

Performance of various neural networks follows a power-law relationship with increasing model and data sizes.

“larger models will continue to perform better” [Kaplan et al., 2020]



GPT-1
2018

Parameter count:
117 Million



GPT-3
2020

Parameter count:
175 Billion



GPT-4
2023

Parameter count:
Expected >Trillion

Environmental Impact

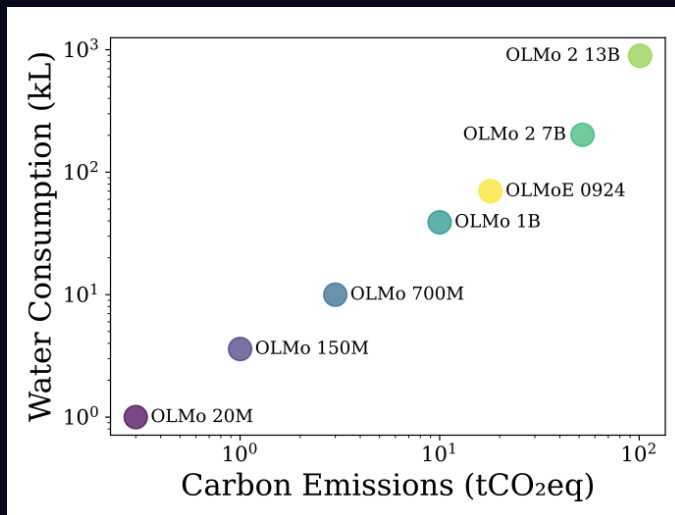
CO ₂ equivalent (g.)					
CT	EIT	EU	Cu	MLCI	GA
237.96	78	0.496	302	290	350.15

“The CO₂ equivalent emissions measured ... greatly depend on the measuring tool used”
(Bannour et al. 2021)

Environmental Impact

CO ₂ equivalent (g.)					
CT	EIT	EU	Cu	MLCI	GA
237.96	78	0.496	302	290	350.15

“The CO₂ equivalent emissions measured ... greatly depend on the measuring tool used”
(Bannour et al. 2021)

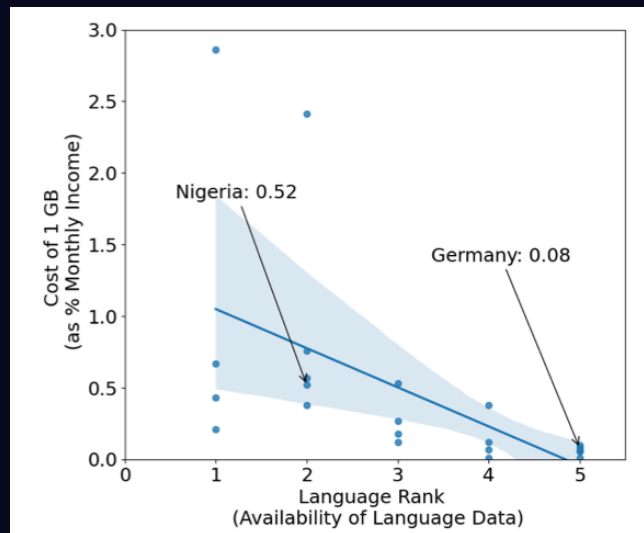


	Power Usage (MWh)	Carbon Emissions (tCO ₂ eq)	Equiv. to... (energy usage, 1 home, U.S.)	Water Consumption (kL)	Equiv. to... (water usage, 1 person, U.S.)
OLMo 2 7B	157	52	10 yrs, 4 mo	202	1 yr, 9 mo
OLMo 2 13B	230	101	21 years	892	7 yrs, 10 mo

Morrison et al., 2025

Accessibility in resource-constrained settings

Ahia et al., 2021



Low-Resource Double Bind

Lightweight Probabilistic Downscaling from a Deterministic Base Model

Inspired by Cachay et al. 2026

U-Net base model, adapted for downscaling

Two-stage training process:

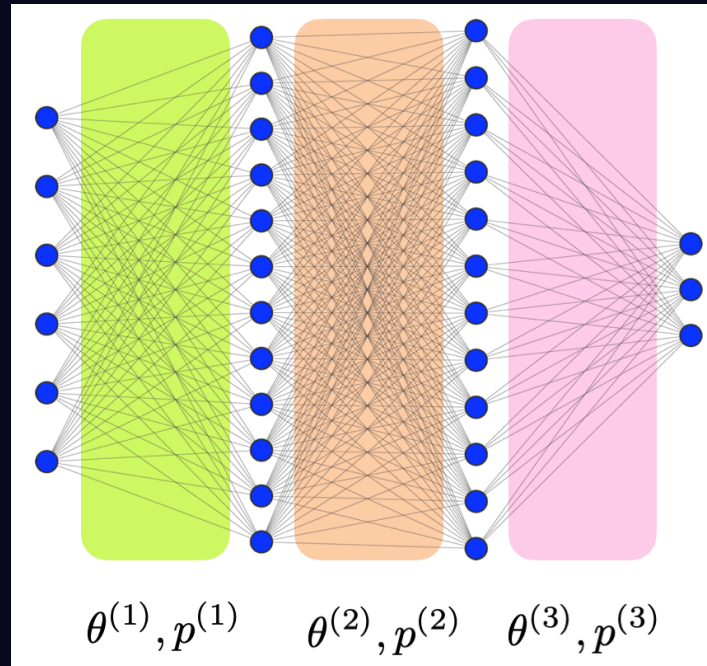
1. Deterministic pre-training
2. CRPS loss and MC dropout to produce ensembles of predictions, allowing the model to learn forecast uncertainties.

CORDEX-ML-Bench (Rampal et al., 2026)

Significantly reduces computational cost.

Preprint: arXiv:2609.29383, McLean et al.

Layer-wise Partitioning of Neural Networks



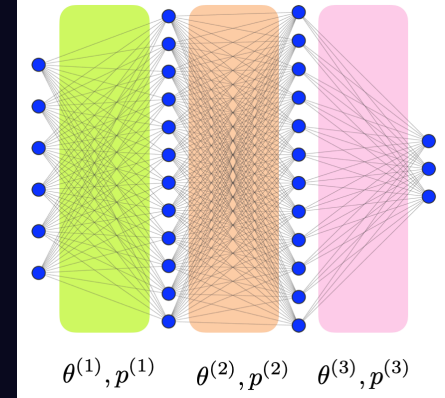
Why?

- Understanding.
- Interpretability.
- Efficiency.
- Improved generalization performance.

Why Partition?

Different network layers play different roles and should be trained differently.

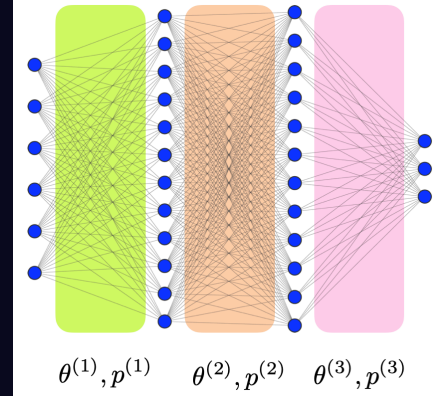
- Early layers capture general features, later layers capture more task-specific knowledge [Yosinski et al., 2014; Hao et al., 2019; Raghu et al., 2019; Neyshabur et al., 2020].
- Layer-wise adaptive learning rates and adaptive moments [You et al., 2017 & 2020].
- Are all layers created equal? [Zhang et al., 2019].
- Different layers are more data-hungry [Bansal et al., 2021].



Why Partition?

Different network layers play different roles and should be trained differently.

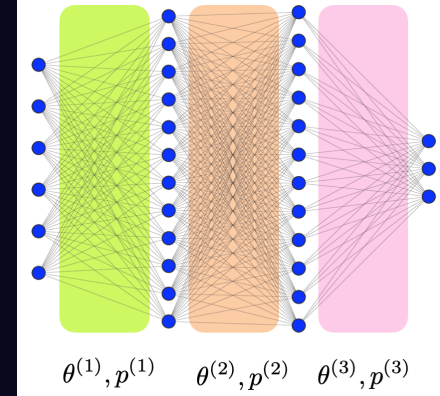
- Early layers capture general features, later layers capture more task-specific knowledge [Yosinski et al., 2014; Hao et al., 2019; Raghu et al., 2019; Neyshabur et al., 2020].
- Layer-wise adaptive learning rates and adaptive moments [You et al., 2017 & 2020].
- Are all layers created equal? [Zhang et al., 2019].
- Different layers are more data-hungry [Bansal et al., 2021].



Why Partition?

Different network layers play different roles and should be trained differently.

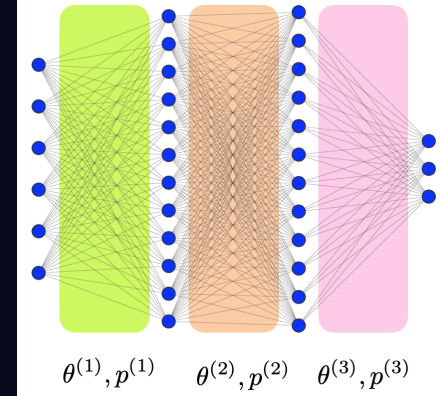
- Early layers capture general features, later layers capture more task-specific knowledge [Yosinski et al., 2014; Hao et al., 2019; Raghu et al., 2019; Neyshabur et al., 2020].
- Layer-wise adaptive learning rates and adaptive moments [You et al., 2017 & 2020].
- Are all layers created equal? [Zhang et al., 2019].
- Different layers are more data-hungry [Bansal et al., 2021].



Why Partition?

Different network layers play different roles and should be trained differently.

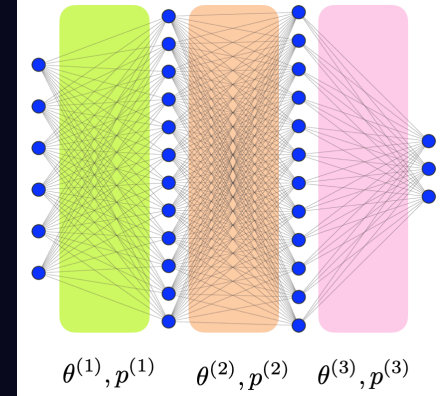
- Early layers capture general features, later layers capture more task-specific knowledge [Yosinski et al., 2014; Hao et al., 2019; Raghu et al., 2019; Neyshabur et al., 2020].
- Layer-wise adaptive learning rates and adaptive moments [You et al., 2017 & 2020].
- Are all layers created equal? [Zhang et al., 2019].
- Different layers are more data-hungry [Bansal et al., 2021].



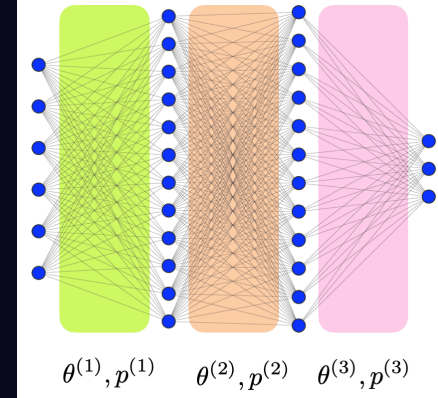
Why Partition?

Different network layers play different roles and should be trained differently.

- Early layers capture general features, later layers capture more task-specific knowledge [Yosinski et al., 2014; Hao et al., 2019; Raghu et al., 2019; Neyshabur et al., 2020].
- Layer-wise adaptive learning rates and adaptive moments [You et al., 2017 & 2020].
- Are all layers created equal? [Zhang et al., 2019].
- Different layers are more data-hungry [Bansal et al., 2021].



Why Partition?



Different network layers play different roles and should be trained differently.

- Early layers capture general features, later layers capture more task-specific knowledge [Yosinski et al., 2014; Hao et al., 2019; Raghu et al., 2019; Neyshabur et al., 2020].
- Layer-wise adaptive learning rates and adaptive moments [You et al., 2017 & 2020].
- Are all layers created equal? [Zhang et al., 2019].
- Different layers are more data-hungry [Bansal et al., 2021].

Related to this topic:

- Layerwise sensitivity to initialization and optimizer hyperparameters [TV and Frankle, ICML 2022].
- Multirate training of neural networks [TV and Leimkuhler, NeurIPS 2022].
- Normalization layers are all that SAM needs [Müller et al., NeurIPS 2023].
- BatchNorm layers have an outsized effect on adversarial robustness [NeurIPS workshop 2025].
- Could Model Partitioning Make Federated Learning More Sustainable? [Frohlich et al., LOCO 2026].

Inspiration

- Neuroscience: synapses have dynamics at different time-scales.
- Fast/slow weights in machine learning: Hinton and Plaut (1987) set each connection to have fast changing weight [*temporary memory*] and slowly changing weight [*long-term knowledge*].

Inspiration

- Neuroscience: synapses have dynamics at different time-scales.
- Fast/slow weights in machine learning: Hinton and Plaut (1987) set each connection to have fast changing weight [*temporary memory*] and slowly changing weight [*long-term knowledge*].
- Molecular dynamics: fast dynamics cheap to compute [e.g., r-RESPA Tuckerman et al. 1991, 1992] -can we obtain computational speed-up in neural network training applications?

Inspiration

WideResNet-16 trained on patch-augmented [Li, Wei & Ma, NeurIPS 2019] CIFAR-10 data:
20% is patch-free, 16% has only the patch, and the rest has both data and patch.

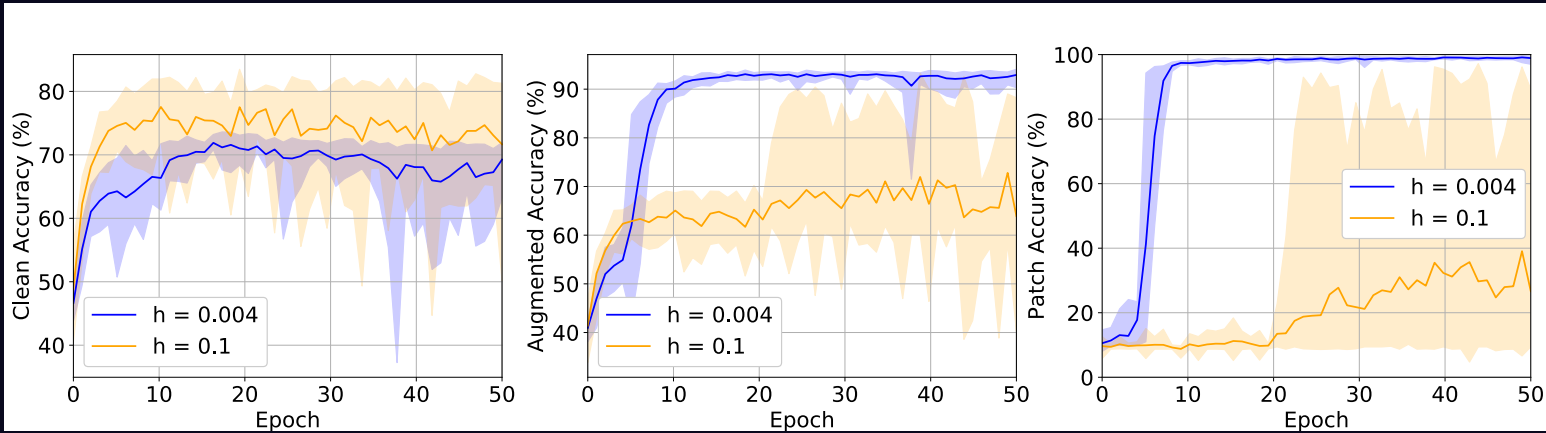


Inspiration

WideResNet-16 trained on patch-augmented [Li, Wei & Ma, NeurIPS 2019] CIFAR-10 data:

20% is patch-free, 16% has only the patch, and the rest has both data and patch.

Optimizer: SGD with momentum with weight decay.



A net trained using a:

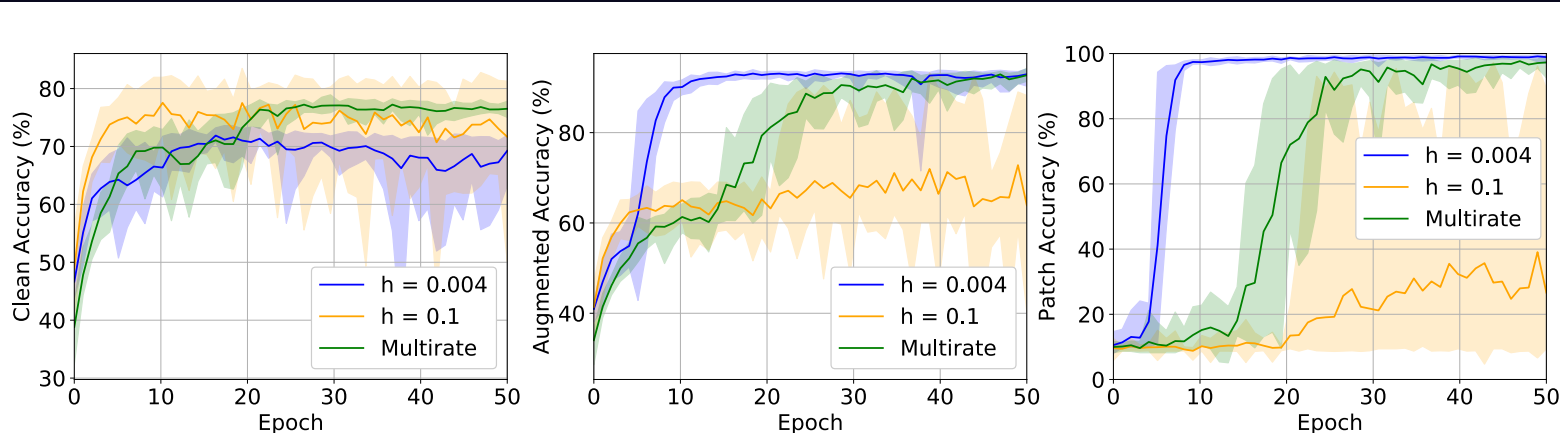
- Small learning rate (blue) memorizes patch.
- Large learning rate (orange) gives higher accuracy on clean data.

Inspiration

WideResNet-16 trained on patch-augmented [Li, Wei & Ma, NeurIPS 2019] CIFAR-10 data:

20% is patch-free, 16% has only the patch, and the rest has both data and patch.

Optimizer: SGD with momentum with weight decay.



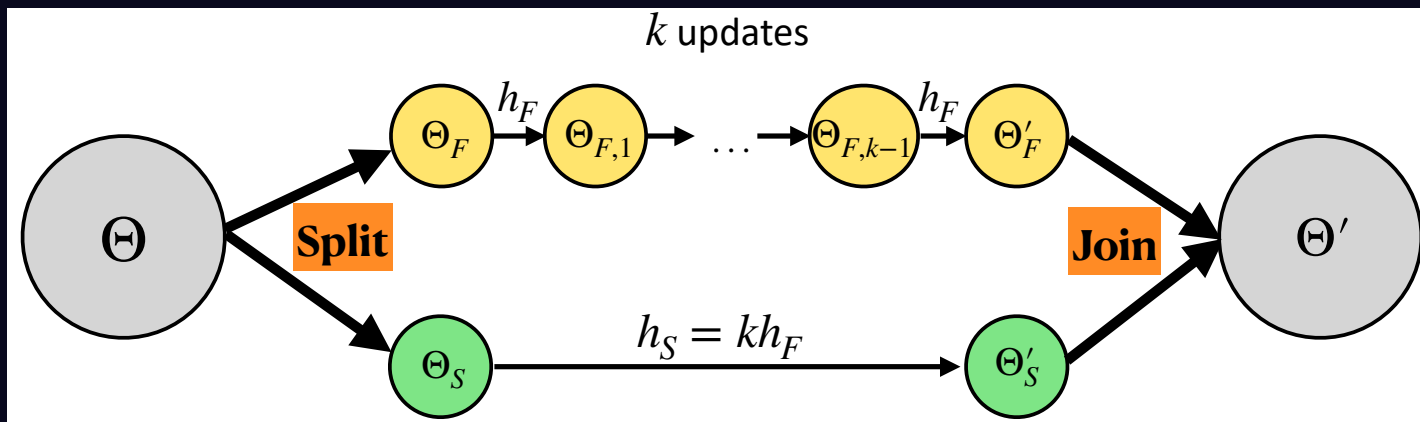
A net trained using a:

- Small learning rate (blue) memorizes patch.
- Large learning rate (orange) gives higher accuracy on clean data.
- Our multirate approach (green) can perform well on both.

Multirate Methods

Two time-scales example:

Partition model (+ accompanying momentum) parameters $\Theta = (\Theta_F, \Theta_S)$.

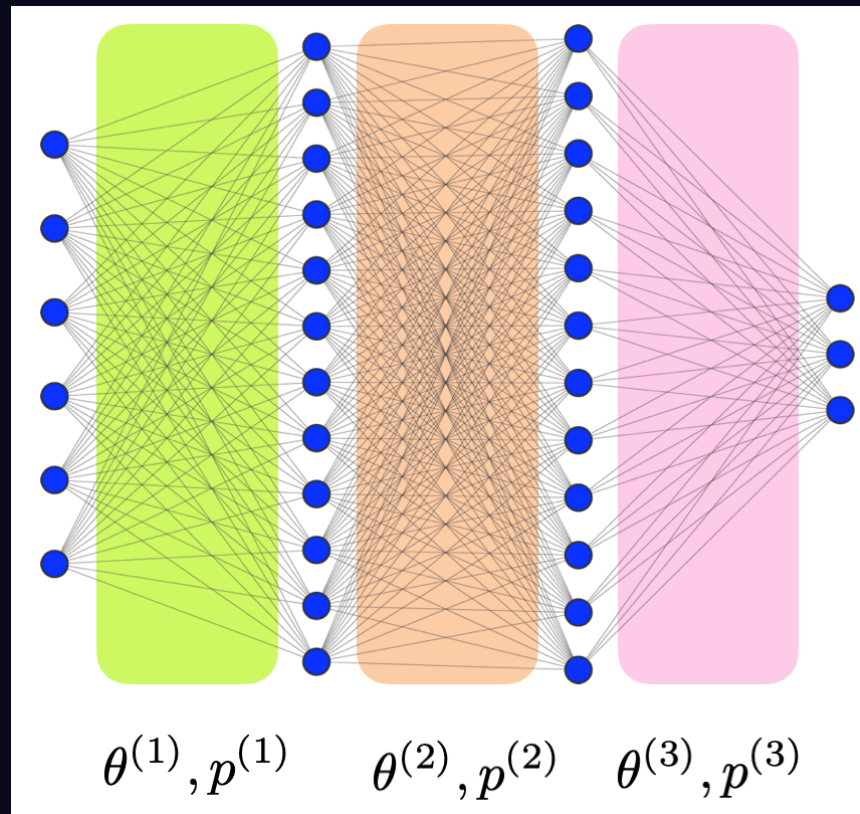


Fast components Θ_F are updated every step with step size h_F
Slow components Θ_S are updated every k steps with step size $h_S = kh_F$

Neural Network Training: Partitioning

Partition model parameters $\theta = (\theta^{(1)}, \dots, \theta^{(N)})$.

Many options.. here we focus on:



Collecting all weights and biases at a given layer of the network.

Partition-based Multirate Approach

Separate neural network parameters into different parts.
You have a choice!

Examples: layer-wise, weight vs. biases, or random subgroups.

Partition-based Multirate Approach

Separate neural network parameters into different parts.
You have a choice!

Examples: layer-wise, weight vs. biases, or random subgroups.

Act as an add-on to existing optimization schemes

If base algorithm SGD:

```

$$\begin{aligned} p_S &:= \mu p_S + \nabla_{\theta_S} \mathcal{L}(\theta_S, \theta_F) \\ \theta_S &:= \theta_S - h p_S \\ \textbf{for } i = 1, 2, \dots, k \textbf{ do} \\ &\quad p_F := \mu p_F + \nabla_{\theta_F} \mathcal{L}(\theta_S, \theta_F) \\ &\quad \theta_F := \theta_F - \frac{h}{k} p_F \\ \textbf{end} \end{aligned}$$

```

with momentum p and loss $\mathcal{L}$

Partition-based Multirate Approach

Separate neural network parameters into different parts.
You have a choice!

Examples: layer-wise, weight vs. biases, or random subgroups.

Act as an add-on to existing optimization schemes

If base algorithm SGD:

```

$$\begin{aligned} p_S &:= \mu p_S + \nabla_{\theta_S} \mathcal{L}(\theta_S, \theta_F) \\ \theta_S &:= \theta_S - h p_S \\ \textbf{for } i = 1, 2, \dots, k \textbf{ do} \\ & \quad p_F := \mu p_F + \nabla_{\theta_F} \mathcal{L}(\theta_S, \theta_F) \\ & \quad \theta_F := \theta_F - \frac{h}{k} p_F \\ \textbf{end} \end{aligned}$$

```

with momentum p and loss $\mathcal{L}$

Partition-based Multirate Approach

Separate neural network parameters into different parts.
You have a choice!

Examples: layer-wise, weight vs. biases, or random subgroups.

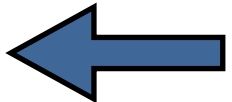
Act as an add-on to existing optimization schemes

If base algorithm SGD:

Without linear drift:

```
 $p_S := \mu p_S + \nabla_{\theta_S} \mathcal{L}(\theta_S, \theta_F)$   
 $\theta_S := \theta_S - h p_S$   
for  $i = 1, 2, \dots, k$  do  
   $p_F := \mu p_F + \nabla_{\theta_F} \mathcal{L}(\theta_S, \theta_F)$   
   $\theta_F := \theta_F - \frac{h}{k} p_F$   
end
```

With linear drift

```
 $p_S := \mu p_S + \nabla_{\theta_S} \mathcal{L}(\theta_S, \theta_F)$   
for  $i = 1, 2, \dots, k$  do  
   $p_F := \mu p_F + \nabla_{\theta_F} \mathcal{L}(\theta_S, \theta_F)$   
   $\theta_F := \theta_F - \frac{h}{k} p_F$   
   $\theta_S := \theta_S - \frac{h}{k} p_S$    
end
```

with momentum p and loss $\mathcal{L}$

Application: Transfer Learning

Basics of transfer learning:

- Start with pre-trained model on large datasets, e.g., ImageNet.
- Remove task-specific layers and re-train (part of) network on new task.

Application: Transfer Learning

Basics of transfer learning:

- Start with pre-trained model on large datasets, e.g., ImageNet.
- Remove task-specific layers and re-train (part of) network on new task.

To obtain computational speed-up:

Split net in two parts: final layer(s) as the fast part, rest is slow part.

Only need to compute gradients for full network every k steps!

Application: Transfer Learning

Basics of transfer learning:

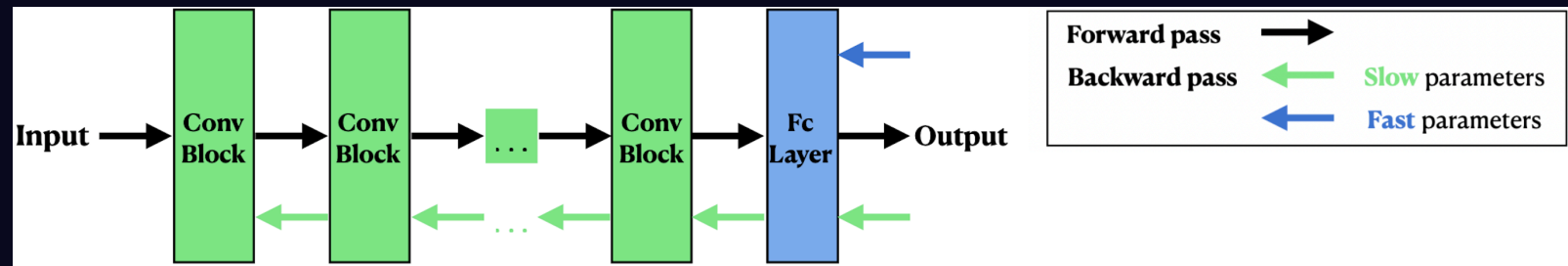
- Start with pre-trained model on large datasets, e.g., ImageNet.
- Remove task-specific layers and re-train (part of) network on new task.

To obtain computational speed-up:

Split net in two parts: final layer(s) as the fast part, rest is slow part.

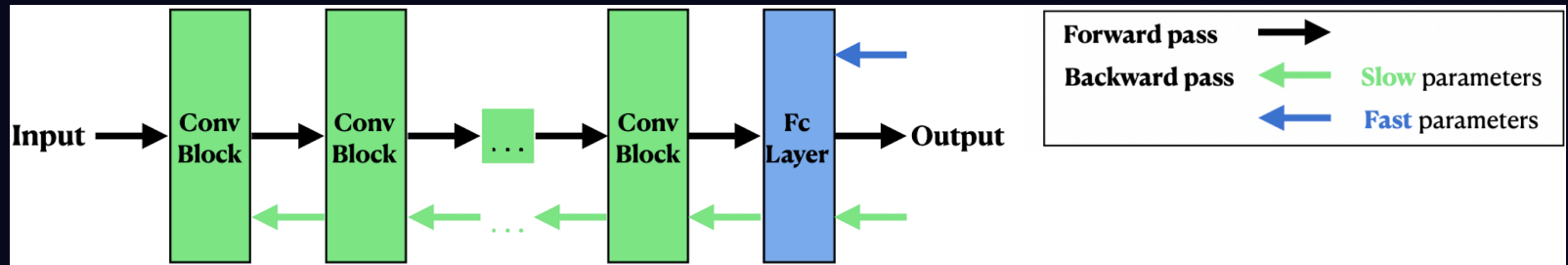
Only need to compute gradients for full network every k steps!

Example for a ResNet architecture:

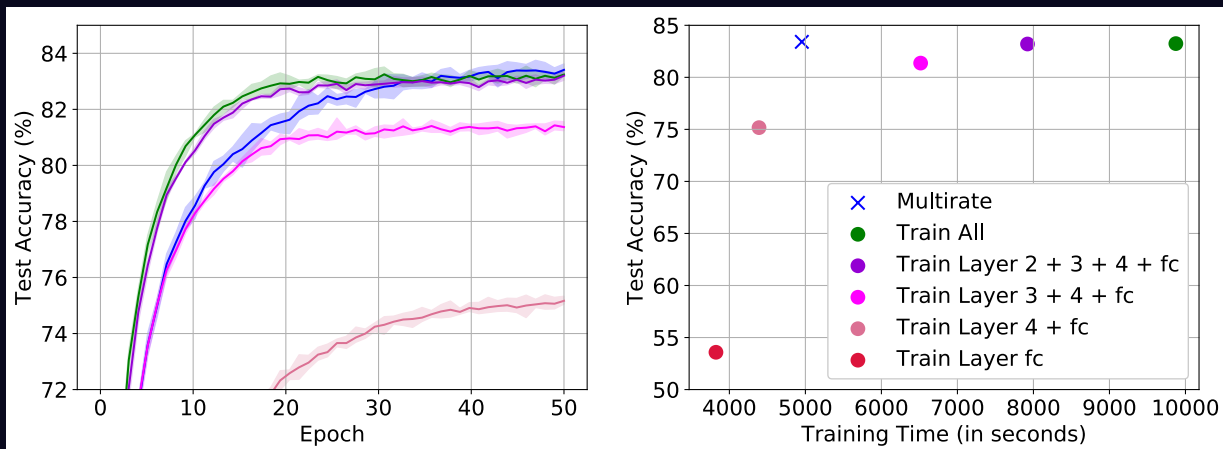


For a ResNet-34 architecture fast parameters are only 0.024% of total.

Application: Transfer Learning

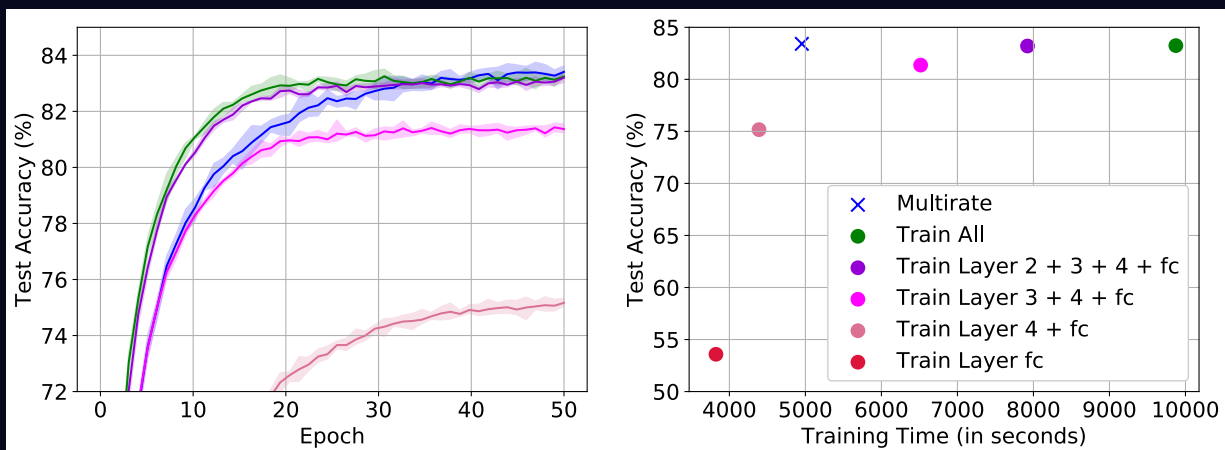


ResNet-50, CIFAR-100 data

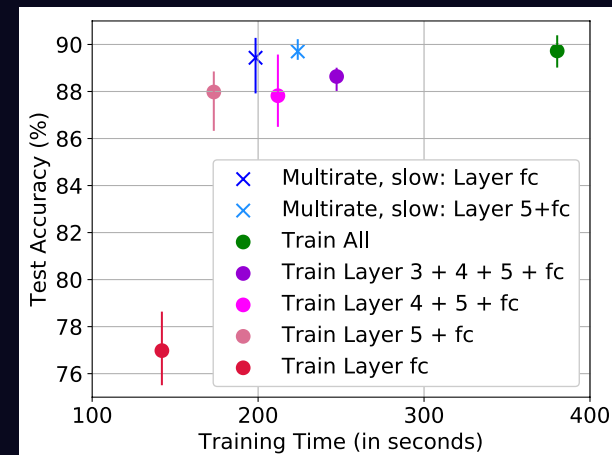


Application: Transfer Learning

ResNet-50, CIFAR-100 data

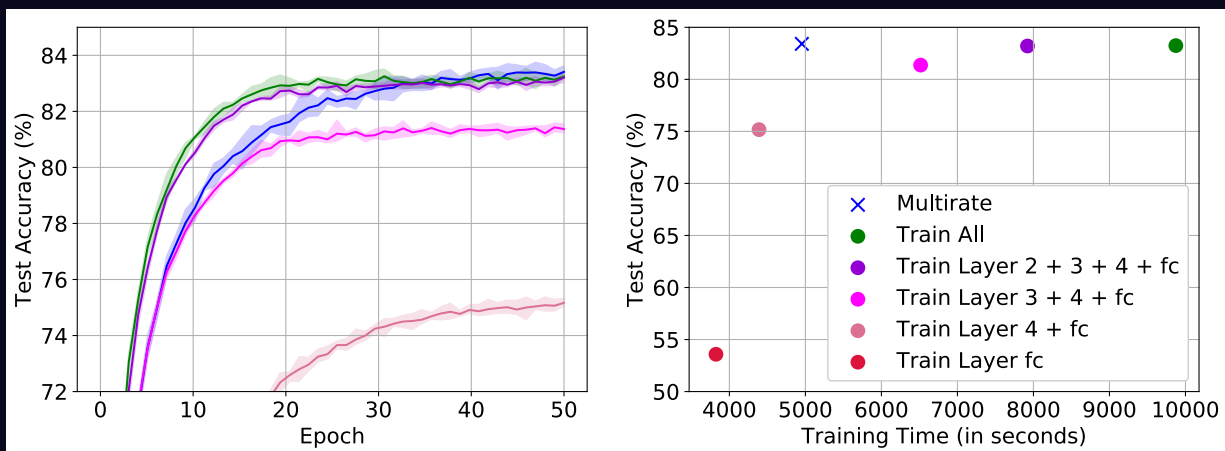


DistilBERT, SST-2 data

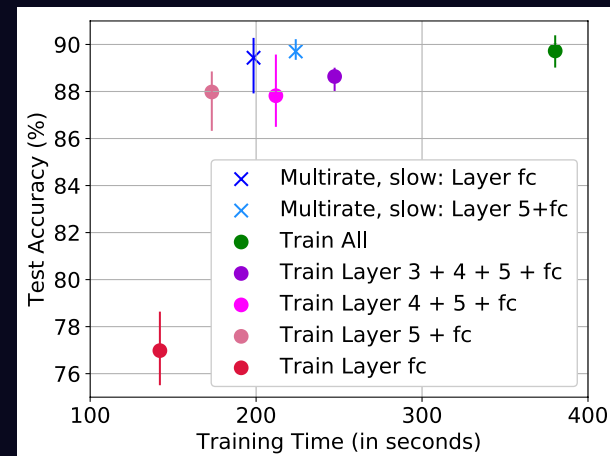


Application: Transfer Learning

ResNet-50, CIFAR-100 data

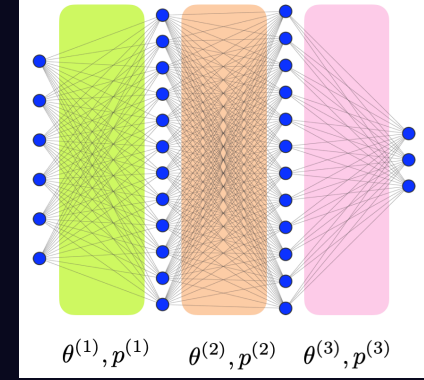


DistilBERT, SST-2 data



Can train in half the time, without losing performance!
Full ablation studies in paper.

Why Partition?



Different network layers play different roles and should be trained differently.

- Early layers capture general features, later layers capture more task-specific knowledge [Yosinski et al., 2014; Hao et al., 2019; Raghu et al., 2019; Neyshabur et al., 2020].
- Layer-wise adaptive learning rates and adaptive moments [You et al., 2017 & 2020].
- Are all layers created equal? [Zhang et al., 2019].
- Different layers are more data-hungry [Bansal et al., 2021].

Related to this topic:

- Layerwise sensitivity to initialization and optimizer hyperparameters [TV and Frankle, ICML 2022].
- Multirate training of neural networks [TV and Leimkuhler, NeurIPS 2022].
- Normalization layers are all that SAM needs [Müller et al., NeurIPS 2023].
- BatchNorm layers have an outsized effect on adversarial robustness [NeurIPS workshop 2025].
- Could Model Partitioning Make Federated Learning More Sustainable? [Frohlich et al., LOCO 2026].

Background: Normalization Layers

Normalization layers form an essential component of most popular architectures, yet there is no conclusive explanation for their success.

Popular choice: Batch Normalization.

[Batch normalization: Accelerating deep network training by reducing internal covariate shift](#)

63154

2015

S Ioffe, C Szegedy

Background: Normalization Layers

Normalization layers form an essential component of most popular architectures, yet there is no conclusive explanation for their success.

Popular choice: Batch Normalization.

Batch normalization: Accelerating deep network training by reducing internal covariate shift

63154

2015

S Ioffe, C Szegedy

$$N(\mathbf{x}) = \gamma \times \frac{\mathbf{x} - \mu}{\sigma} + \beta$$

- Compute mean and standard deviations across batch dimension.
- 2 learnable affine parameters: γ and β

Background: Normalization Layers

Normalization layers form an essential component of most popular architectures, yet there is no conclusive explanation for their success.

Popular choice: Batch Normalization.

Batch normalization: Accelerating deep network training by reducing internal covariate shift

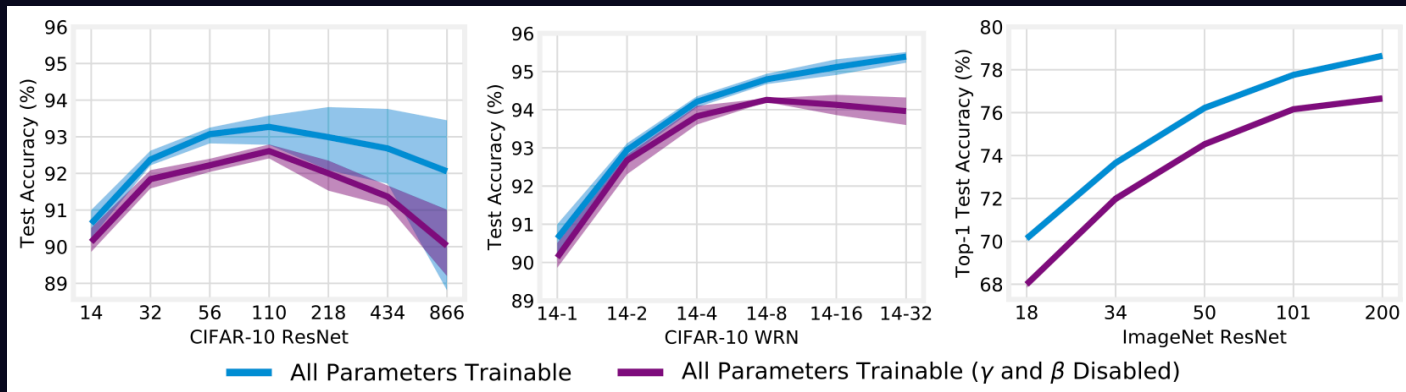
63154

2015

S Ioffe, C Szegedy

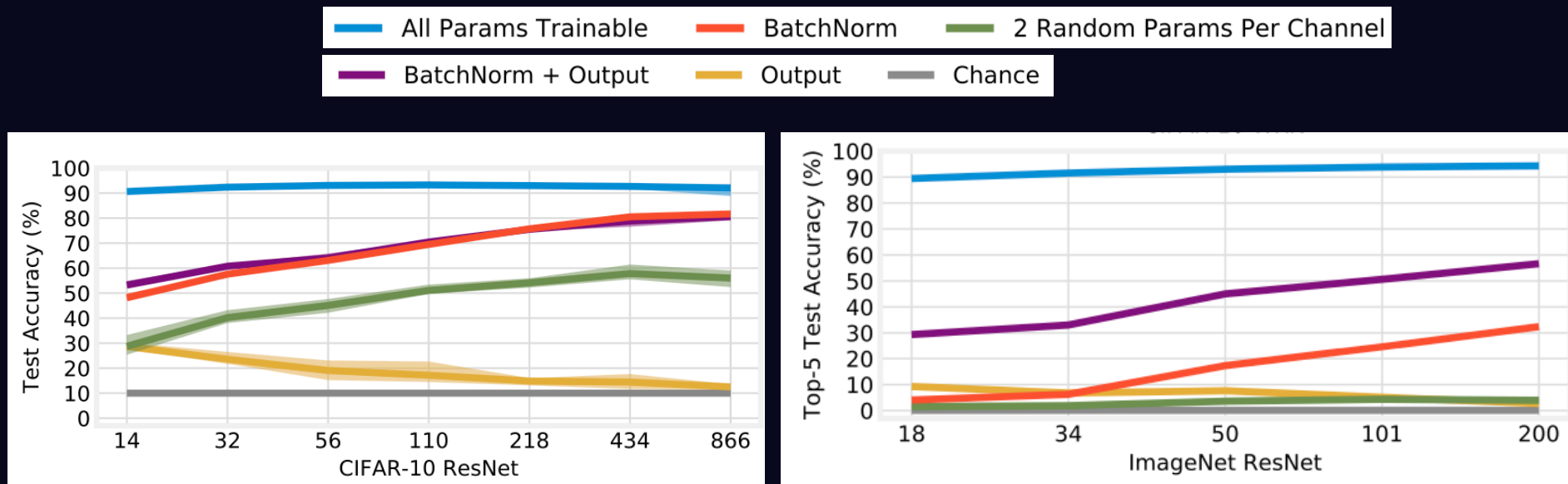
$$N(\mathbf{x}) = \gamma \times \frac{\mathbf{x} - \mu}{\sigma} + \beta$$

- Compute mean and standard deviations across batch dimension.
- 2 learnable affine parameters: γ and β



Frankle et al., 2021

Training BatchNorm and Only BatchNorm



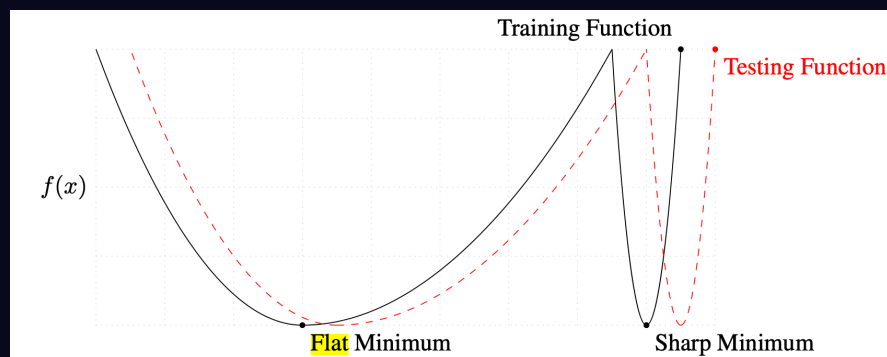
Frankle et al. 2021

Training only the affine normalization parameters (typically $< 1\%$ of total parameters) leads to surprisingly high performance.

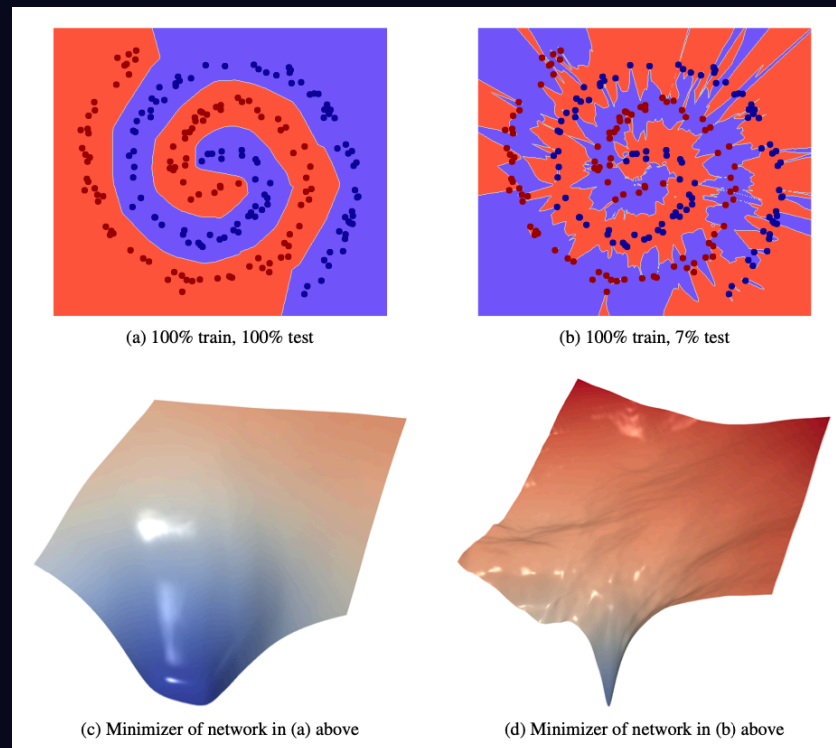
Why Partition? Sharpness-Aware Minimization (SAM)

Background: Flatness and Generalization

Long history. Hochreiter and Schmidhuber, 1997.



“Since flat minimizers can be specified with lower precision than sharp minimizers, they tend to have better generalization performance”, Keskar et al., 2017.



Huang et al., 2020

Some critiques... e.g., Dinh et al., 2017, Andriushchenko et al. 2023.

“sharper minima can generalize better”.

Sharpness-Aware Minimization (SAM)

Foret et al., 2021

Change the objective. Find minima that have both low loss and low sharpness.

$$\min_w \max_{\|\epsilon\|_2 \leq \rho} L(w + \epsilon)$$

Sharpness-Aware Minimization (SAM)

Foret et al., 2021

Change the objective. Find minima that have both low loss and low sharpness.

$$\min_w \max_{\|\epsilon\|_2 \leq \rho} L(w + \epsilon) \quad \max_{\|\epsilon\|_2 \leq \rho} L(w + \epsilon) - L(w) + L(w)$$

Sharpness-Aware Minimization (SAM)

Foret et al., 2021

Change the objective. Find minima that have both low loss and low sharpness.

$$\min_w \max_{\|\epsilon\|_2 < \rho} L(w + \epsilon)$$

Sharpness-Aware Minimization (SAM)

Foret et al., 2021

Change the objective. Find minima that have both low loss and low sharpness.

$$\min_w \max_{\|\epsilon\|_2 \leq \rho} L(w + \epsilon)$$

Additional forward-backward pass.

Sharpness-Aware Minimization (SAM)

Foret et al., 2021

Change the objective. Find minima that have both low loss and low sharpness.

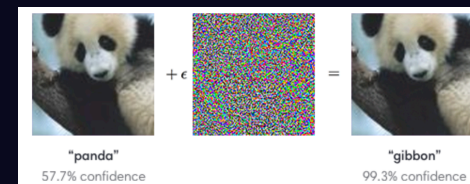
$$\min_w \max_{\|\epsilon\|_2 < \rho} L(w + \epsilon)$$

Additional forward-backward pass.

Works remarkably well !

e.g. SGD + SAM can effectively train Vision Transformers (Chen et al., 2022)

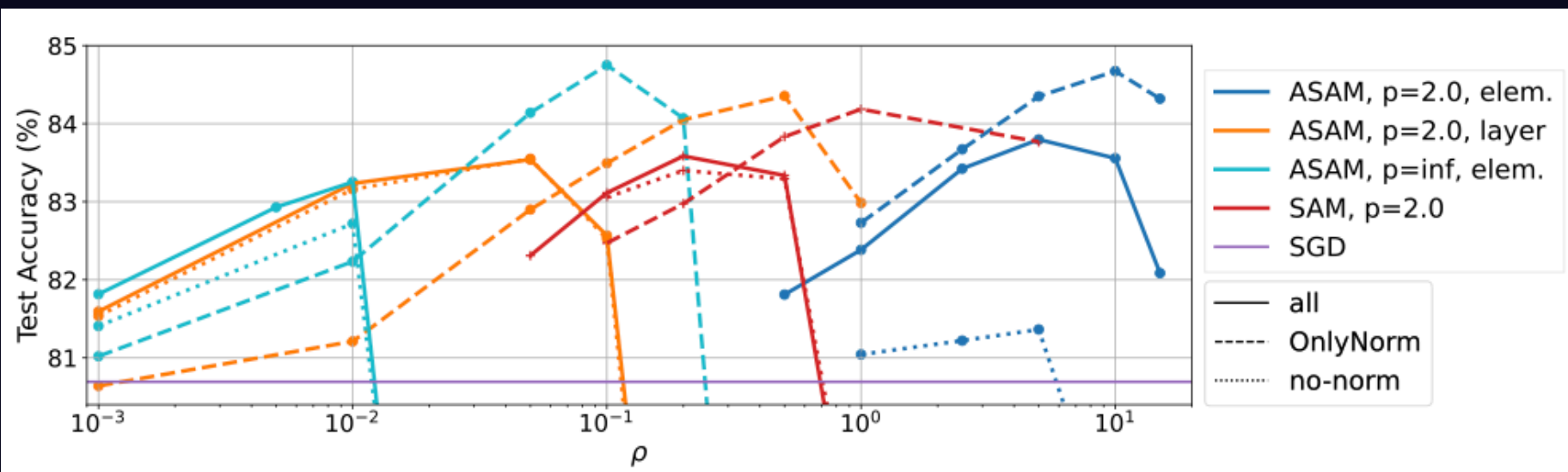
Can also enhance adversarial robustness (Zhang et al., 2024)



Why Partition? Sharpness-Aware Minimization (SAM)

Perturbing *only the normalization layers* (less than 0.1% of the total parameters) in the adversarial step of SAM (SAM-OnlyNorm) outperforms perturbing *all* parameters

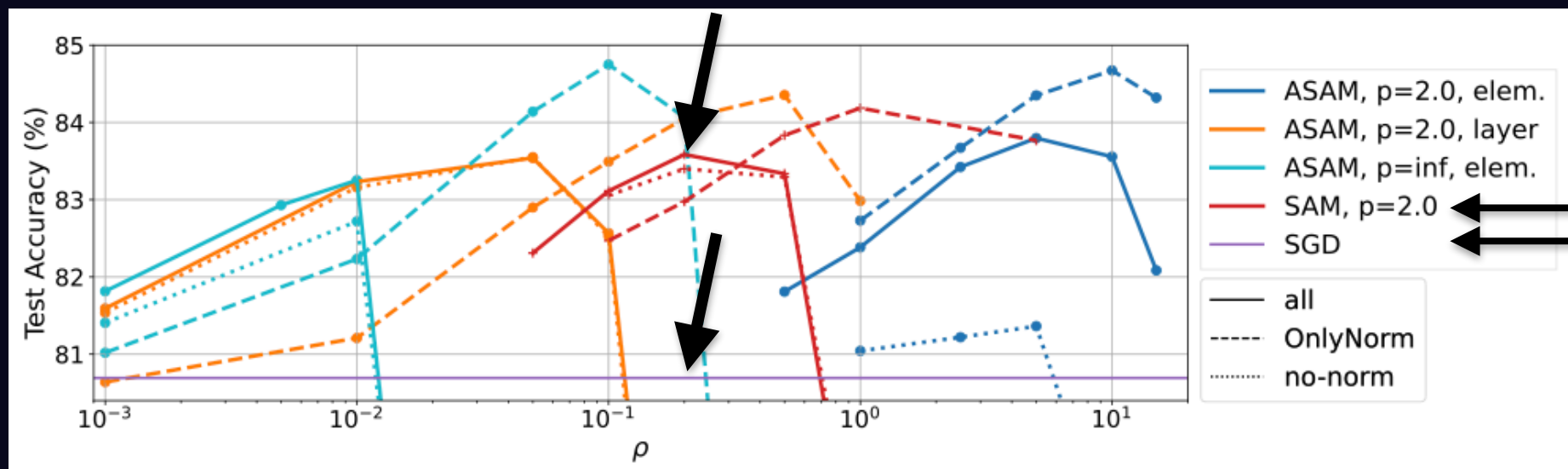
WideResNet-28, CIFAR-100



Why Partition? Sharpness-Aware Minimization (SAM)

Perturbing *only the normalization layers* (less than 0.1% of the total parameters) in the adversarial step of SAM (SAM-OnlyNorm) outperforms perturbing *all* parameters

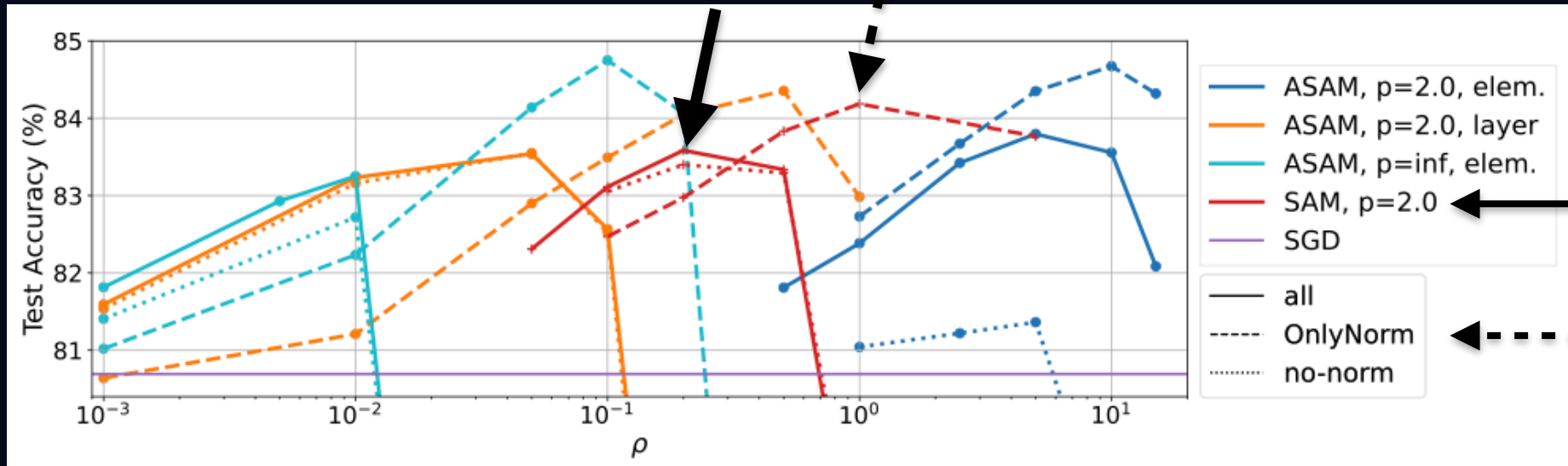
WideResNet-28, CIFAR-100



Why Partition? Sharpness-Aware Minimization (SAM)

Perturbing *only the normalization layers* (less than 0.1% of the total parameters) in the adversarial step of SAM (*SAM-OnlyNorm*) outperforms perturbing *all* parameters

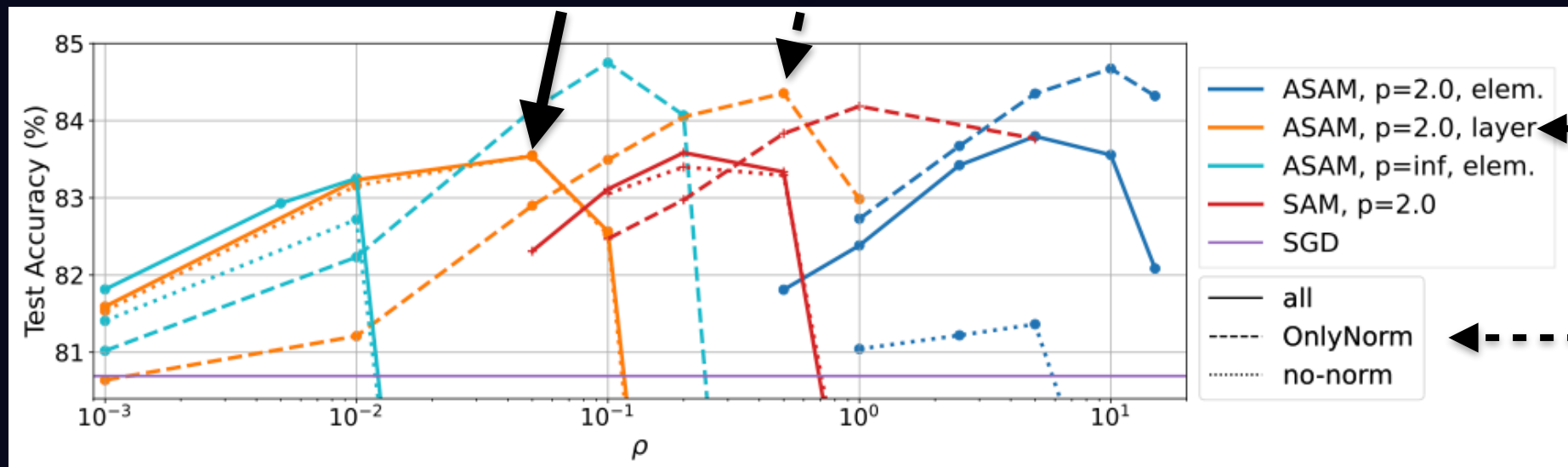
WideResNet-28, CIFAR-100



Why Partition? Sharpness-Aware Minimization (SAM)

Perturbing *only the normalization layers* (less than 0.1% of the total parameters) in the adversarial step of SAM (SAM-OnlyNorm) outperforms perturbing *all* parameters

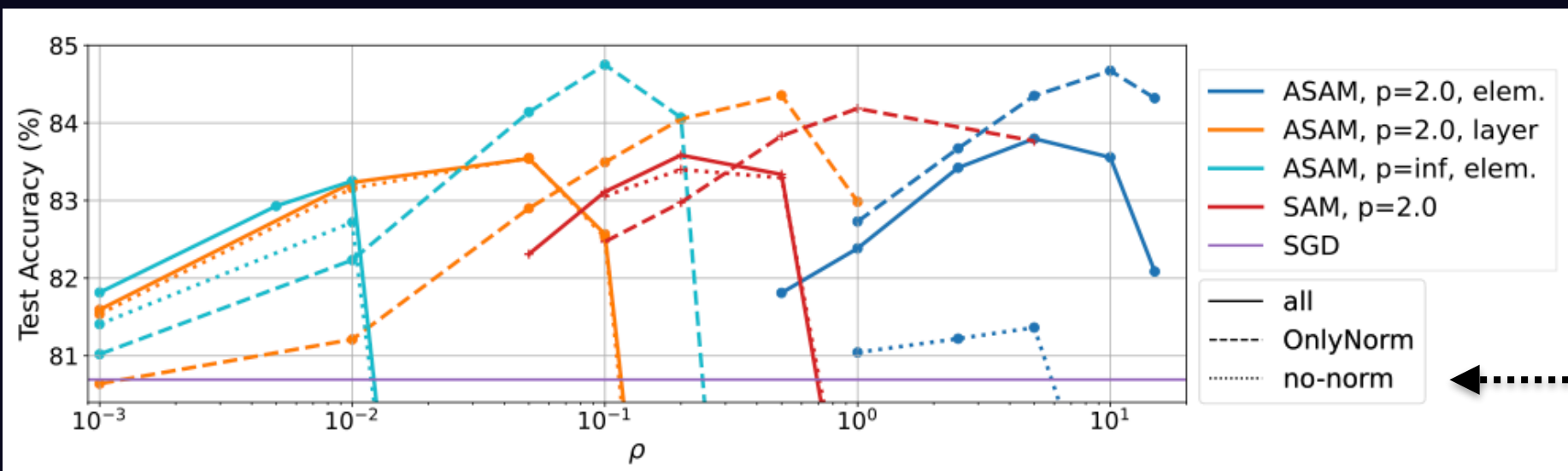
WideResNet-28, CIFAR-100



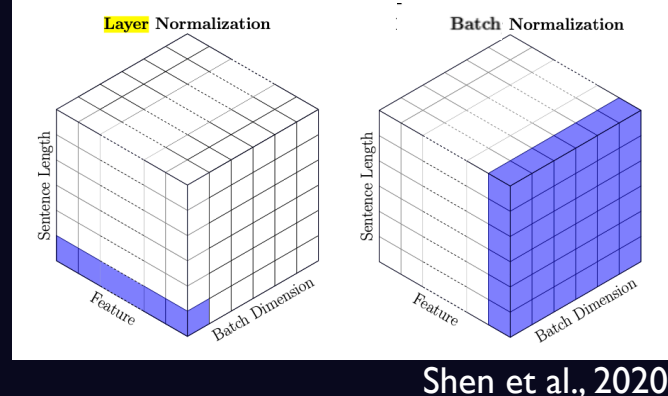
Why Partition? Sharpness-Aware Minimization (SAM)

Perturbing *only the normalization layers* (less than 0.1% of the total parameters) in the adversarial step of SAM (SAM-OnlyNorm) outperforms perturbing *all* parameters

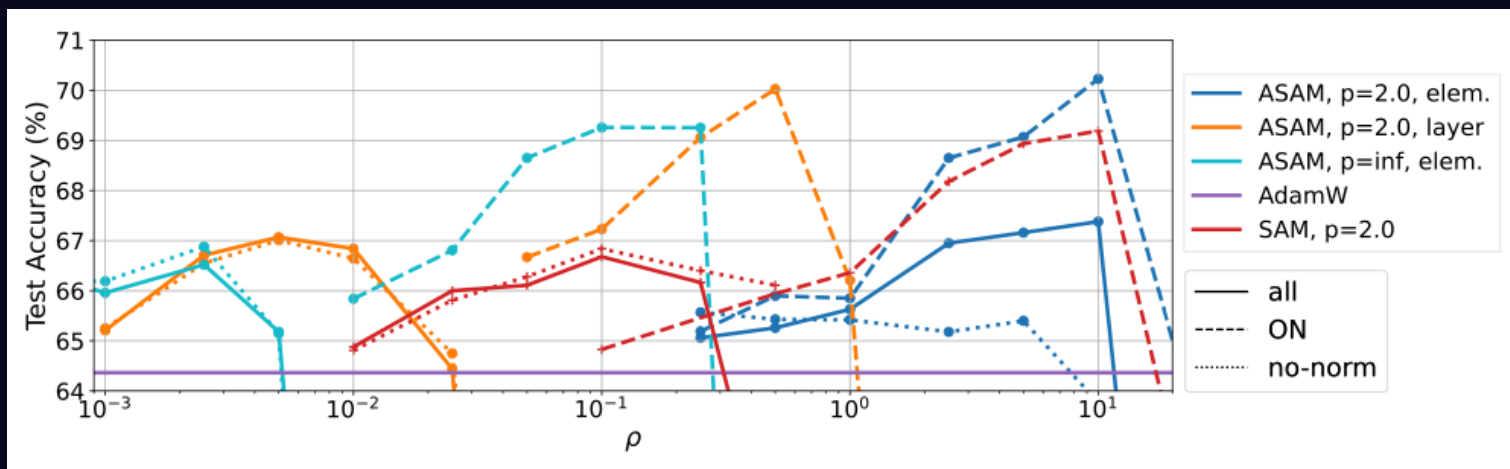
WideResNet-28, CIFAR-100



Works for different settings:



Vision Transformers and LayerNorm



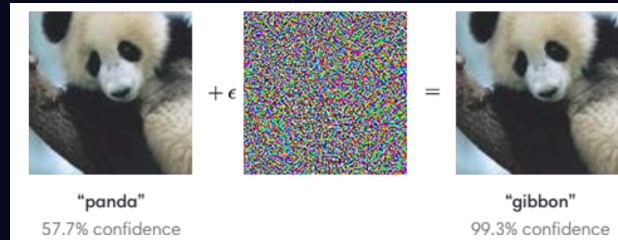
ViT, CIFAR-100

Role of Sharpness

	SGD	SAM-all	SAM-ON
Test Accuracy (%)	80.71 $\pm$ 0.2	83.11 $\pm$ 0.3	84.19 $\pm$ 0.2
ℓ_∞ -sharp. 20 steps, $\rho = 0.003$	0.071 $\pm$ 0.000	0.048 $\pm$ 0.001	0.090 $\pm$ 0.005
20 steps, $\rho = 0.007$	0.433 $\pm$ 0.002	0.309 $\pm$ 0.011	0.585 $\pm$ 0.018
1 step, $\rho = 0.01$	0.204 $\pm$ 0.005	0.183 $\pm$ 0.002	0.315 $\pm$ 0.010
1 step, $\rho = 0.03$	0.809 $\pm$ 0.003	0.769 $\pm$ 0.017	0.843 $\pm$ 0.007

WideResNet-28, CIFAR-100

SAM-ON models are sharper, yet generalize better.



Why Partition? Adversarial Robustness.

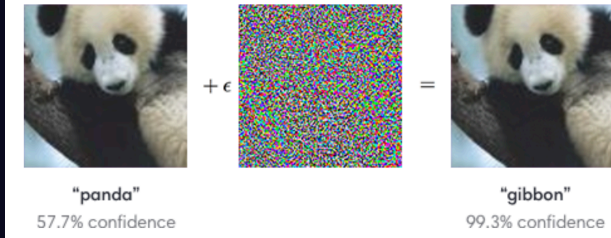
	ϵ	AdamW			Lion		
		vanilla	SAM-all	SAM-ON	vanilla	SAM-all	SAM-ON
l_2	0.25	19.67 ± 0.47	37.53 ± 0.69	41.16 ± 0.24	22.01 ± 0.78	38.52 ± 0.66	43.12 ± 0.97
l_2	0.50	5.47 ± 0.18	17.71 ± 0.61	22.72 ± 0.25	6.63 ± 0.46	19.03 ± 0.92	24.27 ± 1.34
l_2	1.00	0.43 ± 0.09	3.34 ± 0.36	5.58 ± 0.19	0.57 ± 0.07	3.98 ± 0.28	6.64 ± 0.69
l_∞	0.25/255	33.45 ± 0.80	48.08 ± 0.14	49.34 ± 0.08	35.31 ± 0.08	49.57 ± 0.60	51.37 ± 0.99
l_∞	0.5/255	14.98 ± 0.18	29.68 ± 0.09	32.46 ± 0.15	15.86 ± 0.13	31.68 ± 0.62	34.23 ± 1.73
l_∞	1/255	2.61 ± 0.16	8.64 ± 0.01	10.82 ± 0.56	2.93 ± 0.29	10.02 ± 0.56	12.03 ± 1.30
Clean acc.		66.89 ± 0.04	71.47 ± 0.12	71.37 ± 0.026	68.20 ± 0.02	71.90 ± 0.19	72.64 ± 0.14

ImageNet, ViT-S trained from scratch

To get SAM-like improvements for adversarial robustness it is enough to only perturb the normalization layers (SAM-ON).

to only perturb the normalization layers in SAMh.

Why Partition? Adversarial Robustness.

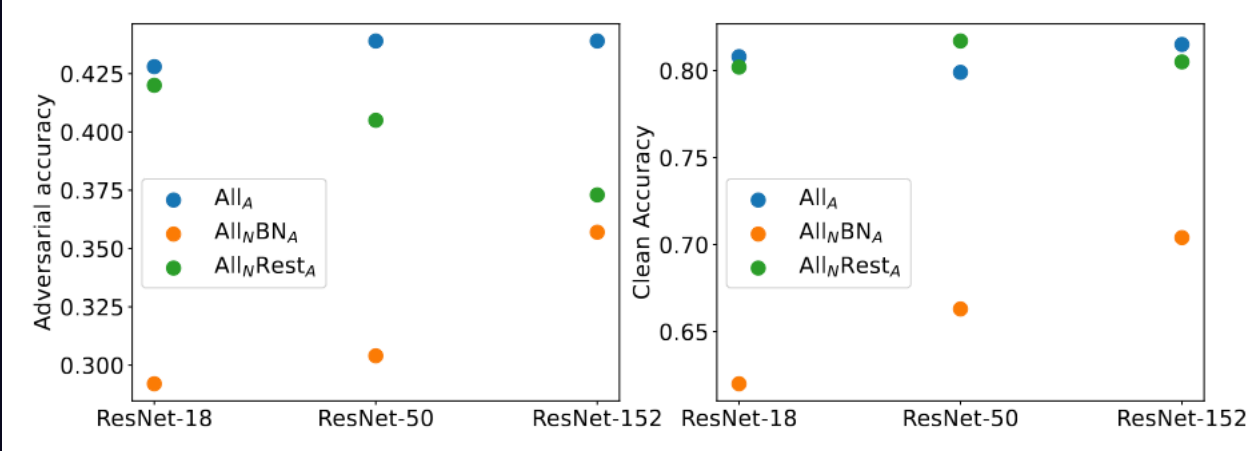


	ϵ	vanilla	AdamW		vanilla	Lion	
			SAM-all	SAM-ON		SAM-all	SAM-ON
l_2	0.25	19.67 ± 0.47	37.53 ± 0.69	41.16 ± 0.24	22.01 ± 0.78	38.52 ± 0.66	43.12 ± 0.97
l_2	0.50	5.47 ± 0.18	17.71 ± 0.61	22.72 ± 0.25	6.63 ± 0.46	19.03 ± 0.92	24.27 ± 1.34
l_2	1.00	0.43 ± 0.09	3.34 ± 0.36	5.58 ± 0.19	0.57 ± 0.07	3.98 ± 0.28	6.64 ± 0.69
l_∞	0.25/255	33.45 ± 0.80	48.08 ± 0.14	49.34 ± 0.08	35.31 ± 0.08	49.57 ± 0.60	51.37 ± 0.99
l_∞	0.5/255	14.98 ± 0.18	29.68 ± 0.09	32.46 ± 0.15	15.86 ± 0.13	31.68 ± 0.62	34.23 ± 1.73
l_∞	1/255	2.61 ± 0.16	8.64 ± 0.01	10.82 ± 0.56	2.93 ± 0.29	10.02 ± 0.56	12.03 ± 1.30
Clean acc.		66.89 ± 0.04	71.47 ± 0.12	71.37 ± 0.026	68.20 ± 0.02	71.90 ± 0.19	72.64 ± 0.14

ImageNet, ViT-S trained from scratch

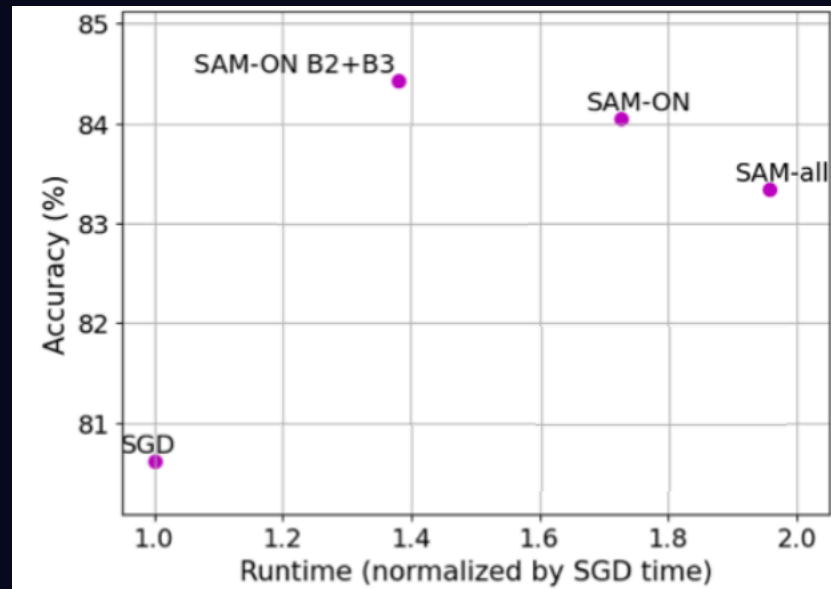
To get SAM-like improvements for adversarial robustness it is enough to only perturb the normalization layers (SAM-ON).

to only perturb the normalization layers in SAMh.



Gap between adversarially trained model (blue, All_A) and a model where only the BatchNorm layers are adversarially re-trained (orange, All_{NBN_A}) decreases with model depth (Zeise and TV, 2025).

Why Partition? Computational Gains.



WideResNet-28, CIFAR-100

Computational gains (and enhanced accuracy) obtained when only perturbing selected normalization layers.

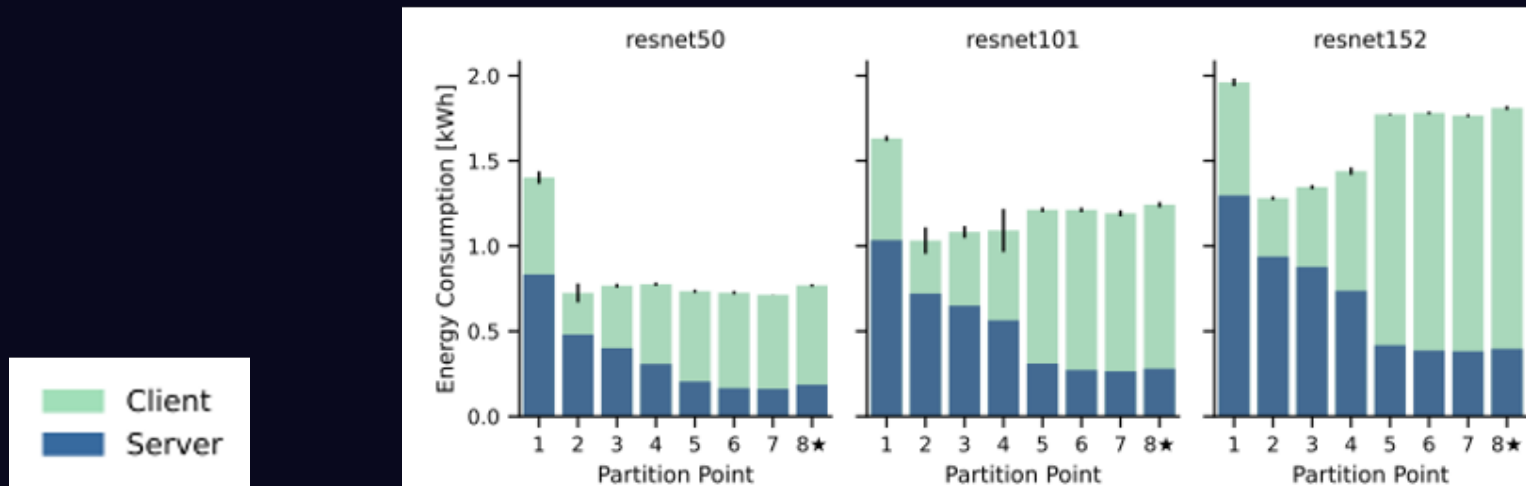
Could Model Partitioning Make Federated Learning More Sustainable?

Led by PhD student Tobias Frohlich
LOCO 2026

Federated learning: models are trained collaboratively across a distributed network of devices.

Model partitioning splits neural networks and offloads a part to the server to reduce client-side computation.

Reduce federated learning operational carbon emissions by performing model partitioning in response to local electricity grid conditions and on-site energy availability to optimise the use of low-carbon energy.



Continue with Tutorial 2

After break: Final Tutorial

Can already start loading in CIFAR-10 data before
the break as this is slow